Open Access | https://doi.org/10.25080/akyc5426



SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Automated Data Enrichment for Police Accountability: Where Agentic Judgment Earns Its Place

Hongsup Shin[1]

[1]Texas Justice Initiative

Abstract

Automated data enrichment, filling missing fields in structured records from unstructured sources, is the canonical case for pointing an autonomous agent at a database and letting it fill every blank. In high-stakes data that instinct is dangerous. A confidently wrong value is worse than a blank, and retrieval-grounded extraction reduces but does not remove the tendency to assert what the source never stated. An LLM can extract these fields; this paper asks where agentic judgment earns its place and where it becomes a liability.

We study this on the Texas Justice Initiative's police shooting databases, where nearly two thousand records are missing the weapon, the subject's race, or the outcome, whose fields volunteers typically recover by hand, fifteen to thirty minutes each. Our LangGraph pipeline, deterministic in its control flow, searches, validates, extracts, and escalates hard cases to a human. It completes 92% of officer and 70% of civilian records. On an adversarial probe of twenty incidents we fabricated, where the only correct behavior is to refuse, it completes none of the twenty and asserts nothing, escalating every record to a human. The recovery itself came from a fix to the extraction prompt, without any agent. An autonomous agent pointed at the same fabricated incidents, with more freedom, commits a wrong-article fabrication the pipeline escalates.

If the deterministic core does the recovering, the agentic layer earns its place by making those recovered values trustworthy. Agency lives only in this thin judgment layer above extraction, which holds three bounded judges in two categories. One judge acts on the pipeline's control flow: the relevance judge reads the retrieved articles and, when none actually report this incident, routes the record to a human instead of completing it. The other two judge what extraction produced: the race verifier deletes a value the source never states, and the conflict annotator explains to the reviewer why the sources disagree on a value. Extraction calls an LLM too, but because it only proposes values for these judges to rule on, we do not count it as agentic. Every judge had to clear a reward-hacking-resistant evaluation gate before it shipped. The main contribution of this paper is a discipline, an "earn-it" protocol, for drawing the line between what a high-stakes pipeline should settle deterministically and where it is worth granting agentic judgment.

Keywords police accountability, data enrichment, LangGraph, LLM-as-judge, agentic workflows, evaluation gate, faithfulness, human-in-the-loop

Published Jul 13, 2026

Correspondence to
Hongsup Shin
hongsup.shin@pm.me

Open Access

Copyright © 2026 Shin. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

In Texas, the Office of the Attorney General (OAG) is required by law to collect a report on every officer-involved shooting (OIS) and to publish an annual summary. Those summaries are high-level: they omit age demographics, report dates, and the number of officers involved, and they carry no intersectional analysis [1]. The Texas Justice Initiative (TJI), a

nonprofit, re-publishes the underlying incident records with far more granularity, the detail independent analysis depends on. But the records are incomplete. About a quarter of incidents are filed with an unidentified cause; race is recorded with a coarse vocabulary that law enforcement is known to mischaracterize; weapon and outcome fields are frequently blank [1]. These gaps carry substantive consequences: undercounting and misclassification of police violence are documented at the national scale [2], and each blank field leaves a question about a police shooting that the public record cannot answer.

TJI addresses these gaps through the standard practice of accountability nonprofits: volunteers search news archives one incident at a time, read the coverage, and transcribe the details. The work takes 15-30 minutes per record, and there are nearly two thousand records. Hand-coding at this scale has precedent: Fryer's team extracted every variable from Houston police reports manually after being advised that the natural-language processing of the day could not handle their complexity [3]. This is precisely the kind of tedious, search-and-extract task that modern large language models (LLMs) appear built for, and the temptation is to point an autonomous agent at the database and let it fill every blank. Human-factors research has long documented that operators over-trust automation in exactly such conditions [4]. We test this directly: an autonomous agent on the same adversarial probe declines most incidents but completes one fabricated record with no signal a reviewer could use to distrust it.

This paper argues against that approach for high-stakes domains; that argument, together with a system that embodies the alternative, is its contribution. The failure mode that matters is the *confidently wrong* field, worse than the blank a volunteer would otherwise leave: a fabricated weapon, a misattributed race, a detail lifted from a different shooting that shares a city and a date. Generative extraction grounded in retrieved documents [5] reduces but does not remove *unfaithfulness*, the tendency to emit content the source does not support, a failure documented in summarization [6] and surveyed broadly for LLMs [7], [8]. Uncritical automation therefore recovers and fabricates in the same pass [9]. This leads to the following engineering questions: how to earn the right to trust what an LLM extracts, and how to decide which decisions warrant an LLM.

Our answer is a system designed as a *workflow* with bounded agentic judgment rather than an autonomous agent. The distinction is a useful design axis [10]: a workflow orchestrates models and tools along predefined code paths, whereas an agent lets the model direct its own process, as in ReAct-style loops that interleave reasoning and action under the model's control [11]. Frameworks such as LangGraph [12] make either style straightforward to build as a stateful graph, but autonomy is a cost as much as a capability: every place the model is allowed to choose is a place it can choose wrong. Two senses of *agentic* are worth separating: whether a model *directs control flow* (chooses what runs next), and whether a model *holds authority over a record*, disposing of an already-produced value, not merely proposing one (Table 1). Our orchestration is deterministic in the first sense, a state machine in which no model directs the sequence of operations; the one routing outcome attributable to a model judgment is the relevance judge's veto. What separates the judges from extraction is the second sense: extraction only proposes content, whereas three small bounded judges hold authority to veto, null, or flag what it produced, applying the now-common pattern of using an LLM to judge a narrow, checkable question [13] ("do these articles actually describe this incident?"). Their authority is graduated, from the relevance judge's veto down to the conflict annotator's advisory note. Each judge earned its place by clearing an offline evaluation gate designed to resist the most common way such systems fool their builders: optimizing the headline metric (here, completion rate) at the expense of the true objective (faithfulness). We make three claims that should transfer beyond this case:

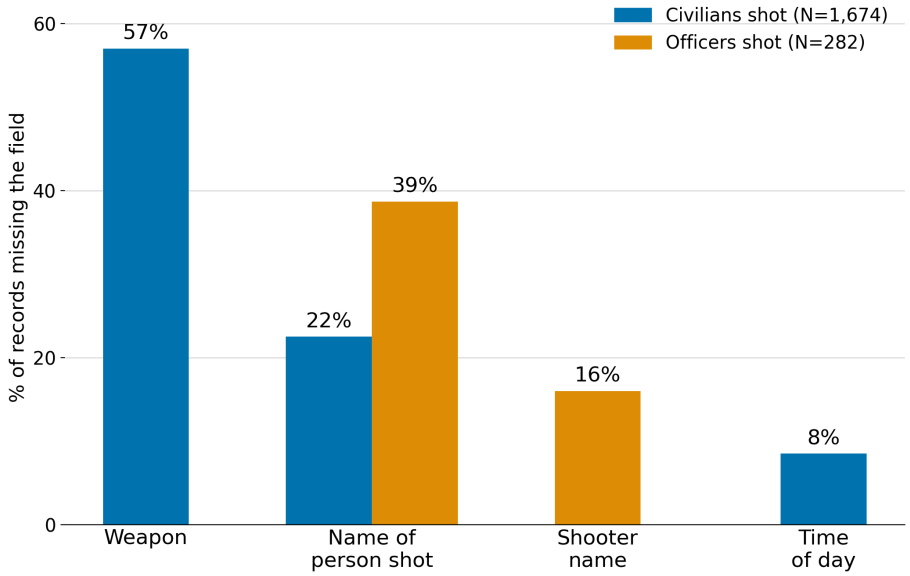
1. **Agentic judgment belongs to the evidence layer, above extraction.** The single largest gain in this project came from a change to the fixed extraction prompt, with no agent involved; the judges add faithfulness and leave recall to retrieval.
2. **A multi-objective, reward-hacking-resistant evaluation gate is the principal safeguard.** Optimizing a proxy metric reliably degrades the true objective once the two diverge [14], [15], and "complete more records" is trivially gamed by accepting weak extractions; the gate therefore scores every change on three gating guards: completion (tracked and surfaced, so a drop is never silent), a hard zero-hallucination veto, and field-level correctness on a stable cohort, measured so that a completion gain cannot launder a correctness loss.
3. **Authority should be calibrated to stakes.** Our three judges can *block* a record into human review, *null* an unsupported value, or merely *advise* a reviewer; they never silently overwrite the record.

The system is open source, and all quantitative results reported below are produced by the pipeline's evaluation harness, applied to the data described next.

2. Methods

2.1. Datasets

TJI publishes two related datasets that we treat throughout: `civilians_shot` (police shooting civilians; 1,674 records) and `officers_shot` (civilians shooting police; 282 records), spanning 2014–2024, for 1,956 records total. They use different field names for analogous concepts (in `officers_shot` the civilian is the *shooter* and the outcome is the *officer's* injury or death), a divergence the pipeline handles through a `DatasetType` enum that switches the database queries and tailors the extraction prompt to each dataset. In the source data, 57% of civilian records are missing the weapon, 22.5% are missing the subject's name, and 39% of officer records are missing the officer's name (Figure 1). The civilian dataset records each involved officer's demographics but no name, so the officer who fired is structurally anonymous in the published record; `officer_name` is one of the fields the pipeline extracts. We measure accuracy against a held-out sample of these records, defined in Evaluation design below.



Single bars: the other dataset does not record the field. Computed from the published TJI tables.

Figure 1. *Missing fields as a share of each dataset's records, computed from the published TJI tables. A single bar means the other dataset does not record that field.*

2.2. Pipeline architecture

The pipeline is a seven-node LangGraph state machine (Figure 2). Each node accepts and returns a single typed state object (a `pydantic` model [16]), and a deterministic **Coordinator** reads the stage that just ran and routes the record to retry, proceed, or escalate:

- **Load** reads incident anchors from PostgreSQL [17] via `psycopg2` [18].
- **Search** queries the Tavily web-search API [19] within a date window around the incident.
- **Validate** keeps only articles that match the incident on a tiered rule: one with a parsed publication date must match on date proximity ($\pm$5 days) and location; one lacking a date must match on location and the victim's name; with neither, it falls back to location alone, keeping date-less results usable without admitting articles that share only a city.
- **Synthesize** runs the LLM extraction and the agentic layer described below.
- **Complete** and **Escalate**, the two terminal nodes, write a JSON result or an escalation report for human review.

When validation leaves too few usable articles, the Coordinator climbs a fixed three-rung search ladder, from an exact-date match to a month-and-year window to a name-only query, and escalates only after the third rung fails.

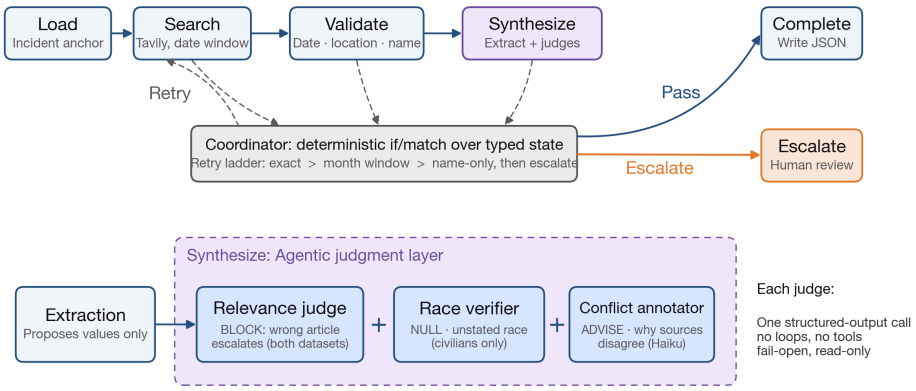


Figure 2. *The seven-node pipeline. The deterministic Coordinator is the hub through which every transition passes: each processing node returns its result to the Coordinator (dashed edges), which dispatches the record onward,* pass *along the happy path (Load, Search, Validate, Synthesize) into the Complete terminal,* retry *back to Search, or* escalate *to the human-review terminal. On thin retrieval it climbs the fixed three-rung search ladder printed on its box before escalating. A retry follows an empty search or a validation that rejects every article; an escalation can originate at any gate. The bottom row expands the Synthesize node: after extraction, which only proposes values, three bounded LLM judges run in sequence as the agentic layer: the relevance judge can* block *a wrong-article completion (the Coordinator then escalates it as* `irrelevant_sources`*), the race verifier* nulls *an unstated race, and the conflict annotator* advises *the human reviewer.*

Each run ends at one of two terminal nodes, and a human reviewer reads the JSON it writes (Figure 3).

```
{
  "incident_id": "178", "dataset_type":
"civilians_shot",
  "extracted_fields": [
    {"field_name": "weapon", "value":
"SHOTGUN", "confidence": "high",
     "sources": ["kwtx.com/content/news/
Officers-cleared-in-deadly..."],
     "source_quotes": ["Ramirez was shot after
he pointed a shotgun
        in the direction of officers."],
     "extraction_method": "llm"}
    // 5 more: civilian_name, civilian_age,
location_detail, time_of_day, outcome
  ],
  "search_strategy": "temporal_expanded",
"retry_count": 1,
  "outcome_summary": "Enriched 6 fields for
incident 178 (civilians_shot)"
}
```

(a)

```
{
  "incident_id": "357", "dataset_type":
"officers_shot",
  "escalation_reason": "irrelevant_sources",
"current_stage": "synthesize",
  "retrieved_articles": [
    {"title": "Capital murder trial of man
accused of killing SAPD officer during 2013
chase begins"}
    // 9 more retrieved articles
  ],
  "outcome_summary": "Escalated incident 357:
irrelevant_sources after 0 retries"
}
```

(b)

Figure 3. *A completed record (`civilians_shot` incident 178):*

An escalated record (`officers_shot` incident 357), vetoed by the relevance judge:

The two terminal outputs (verbatim from the pipeline's saved JSON, abridged; incident identifiers are database keys). The completion (top) carries each field's confidence, sources, and supporting quotes, so a reviewer can verify a value in seconds. The escalation (bottom) is the relevance judge blocking a dossier whose strongest article passes every rule-based check but recounts a different (2013) case.

Two design choices are deliberate. First, **the orchestration is deterministic**: the Coordinator is `if`/`match` logic over state fields and the retry ladder is a fixed list. *Deterministic* here describes the control flow, not the model outputs: every LLM call is genuinely sampled (no temperature is set, so calls use the Anthropic default of 1.0, and the API exposes no seed), so its run-to-run variance, which we return to in Limitations, comes from both sampling and API-side nondeterminism; the architecture is built to catch wrong values, not to remove variance. Which nodes run, in what order, and on which outcomes is fixed code no model can redirect, save the one model-attributable route, the relevance judge's veto. We chose this over an LLM router because, in a high-stakes domain, predictable control flow is itself a safety property. Second, the system is **human-in-the-loop (HITL) by construction**. It never writes back to the source database, and escalation is a designed terminal outcome that hands the record to a human.

The entire graph is assembled in one function (simplified from `src/agents/graph.py`):

```python
def build_graph(checkpointer=None) -> CompiledStateGraph:
    workflow = StateGraph(EnrichmentState)        # state schema = a pydantic model
    workflow.add_node("load", load_node)
    workflow.add_node("search", search_node)
    workflow.add_node("validate", validate_node)
    workflow.add_node("synthesize", synthesize_node)
    workflow.add_node("complete", complete_node)
    workflow.add_node("escalate", escalate_node)
    workflow.add_node("coordinate", coordinate_node)

    workflow.add_edge(START, "load")
    workflow.add_edge("load", "coordinate")       # every processing node
    workflow.add_edge("search", "coordinate")     #   returns to the coordinator hub
    workflow.add_edge("validate", "coordinate")
    workflow.add_edge("synthesize", "coordinate")
    workflow.add_edge("complete", END)
    workflow.add_edge("escalate", END)

    # the only branch in the graph, and it is deterministic (not an LLM router)
    workflow.add_conditional_edges("coordinate", route_after_coordinator)
    return workflow.compile(checkpointer=checkpointer)
```

2.2.1. *How we use LangGraph*

LangGraph models a pipeline as a graph of plain functions over a single typed state object. We use it narrowly: the state schema is the `EnrichmentState` `pydantic` model, so the whole run is one validated, examinable object, and there is exactly one conditional edge, out of the Coordinator, whose target is chosen by a deterministic `match` on the stage that just ran (simplified from `src/agents/coordinate_node.py`):

```python
match state.current_stage:
    case PipelineStage.LOAD:       state = check_load_results(state)
    case PipelineStage.SEARCH:     state = check_search_results(state)
    case PipelineStage.VALIDATE:   state = check_validate_results(state)
    case PipelineStage.SYNTHESIZE: state = check_synthesize_results(state)
# route_after_coordinator(state) then returns state.next_stage.value
```

Where the ecosystem's agent abstractions (LangChain's agent executors, CrewAI, AutoGen) let a model choose tools and coordinate other agents, LangGraph exposes the graph and its typed state directly [12], [20]. The one routing outcome attributable to a model judgment is the relevance judge's veto, which the Coordinator reads like any other state field; its sibling escalations, on empty or conflicting extractions, are model-agnostic completeness guards that would fire the same way for a regex extractor. Dependencies are injected through `RunnableConfig`, so the judges are testable without a live Claude [21] client.

2.3. Deterministic conflict reduction first

Before any LLM is consulted, deterministic code exhausts what rules alone can resolve, so the judges are reserved for what code genuinely cannot do. The steps run in pipeline order:

1. **Aggregation sources are excluded** outright: multi-incident pages such as Wikipedia lists or Fatal Encounters records [22] contaminate extraction by folding several incidents into one document, so the search and validation nodes drop those domains and any PDF or CSV URL, leaving only documents about a single incident.
2. **Extraction is anchored to the record's subject**: when an article describes several people (a second victim, a bystander, the officer), the prompt extracts fields only for the target individual, never blending several together.
3. **Names, race terms, and weapon categories are normalized** before comparison, so that honorific and phrasing variants do not register as disagreements.

4. **A consensus resolver** (`field_normalizers.py`) commits an `outcome` or `time_of_day` value when at least two articles agree on the same canonical form; relatedly, a record can **complete partially**, keeping the fields the sources agree on while routing only a genuinely conflicting field to review.
5. **A race taxonomy** (`race_taxonomy.py`) maps race terms to TJI's buckets for matching while preserving the raw value and flagging divergences for review.

Table 8 shows several of these behaviors on concrete cases.

2.4. Agentic judges with graduated authority

Only after extraction do three LLM judges reason about what rules cannot, each granted authority **calibrated to the stakes** of its decision, from block for the highest-confidence check down to advise for the fuzziest. Every judge is deliberately bounded: a single structured-output call (no loops, no tools), *fail-open* (an error is logged and the pipeline proceeds as if the judge had not run), and read-only with respect to the database. The worst case any judge can produce is one extra human review, never a corrupted record. An outage merely forgoes that judge's extra protection, leaving the record as extraction produced it, still carrying its sources.

Each judge is a typed schema plus a single structured-output call; the relevance judge's is representative (simplified from `src/synthesize/relevance_judge.py`):

```
class RelevanceVerdict(BaseModel):
    relevant_any: bool = Field(description="True if >=1 article reports THIS incident")
    relevant_indices: list[int] = Field(default_factory=list)
    reasoning: str = Field(description="One sentence.")

def judge_relevance(llm_client, state, articles) -> RelevanceVerdict:
    prompt = _build_prompt(state, articles)
    return llm_client.with_structured_output(RelevanceVerdict).invoke(prompt)
```

2.4.1. *Relevance judge: block*

The relevance judge runs on both datasets, asking whether the retrieved articles actually report *this* incident; when they do not, it vetoes the completion and escalates the record as `irrelevant_sources`. It catches the "right structure, wrong incident" failure that rule-based validation cannot. The clearest case is the San Antonio March 2018 officer shooting of Figure 3, where an article published on the incident date in fact recounts a *2013* trial; reading the text, the judge caught what the rules could not, and the record escalated. The same gate closes the civilian famous-name collision, a victim sharing a name with a high-profile case whose coverage is all about the other person (Table 8).

2.4.2. *Race verifier: null*

The race verifier runs on civilians only and asks whether the source *explicitly states* the race of this subject. It is a faithfulness filter: it nulls any `civilian_race` the source does not support, never inferring race from a name, neighborhood, or photo, which mirrors TJI's own caution that race is frequently mischaracterized [1]. The offline audits show what it declines: a "White" whose supporting sentence described a white Chevy pickup, a "Hispanic" inferred from a surname. Its restraint is visible on the holdout: of the eleven civilian races it committed, only one disagreed with the database, recording Black where the database says Hispanic.

2.4.3. *Conflict annotator: advise*

The conflict annotator, run on the cheaper Claude Haiku model across both datasets, writes an advisory triage note when a deep conflict reaches a human, explaining *why* the sources disagree (a lawsuit and a police report diverging on the weapon and circumstances, for instance). Because it never commits a value, it cannot change accuracy or coverage by construction; its only effect is to speed the reviewer.

Building these judges taught design lessons that generalize. Date works as a *hard* gate at two granularities: the Validate node checks the article's *publication* date, the relevance judge the date of the *event it describes*. The recorded outcome is only *supporting* context, because the database's own harm field is sometimes stale. Cost discipline runs throughout: high-frequency or simple calls use Haiku, rare or complex ones use Claude Sonnet [21].

Table 1. *Where agency lives. Each component is classified by the two senses of* agentic*: does a model direct control flow (a model-attributable routing outcome), and does a model hold authority over the record (veto, null, or flag a produced value)? Four components call an LLM, yet only the three judges hold authority and only the relevance judge reaches control flow; extraction calls an LLM but only proposes content, so it is non-agentic despite that call.*

Component	LLM?	Directs control flow?	Holds authority?	Class
Relevance judge	Yes	Yes (routes to escalation)	Yes (block)	Agentic, both senses
Race verifier	Yes	No	Yes (null)	Agentic, authority
Conflict annotator	Yes (Haiku)	No	Yes (advise)	Agentic, authority
Extraction	Yes	No	No (proposes)	Not agentic
Orchestration, conflict reduction, evaluation gate	No	No	No	Deterministic

Table 2 shows how often each route fires across the two 100-record held-out samples and the 20-incident adversarial probe (both detailed below). Most non-completions are deterministic retrieval gaps; the relevance-judge veto accounts for a small but real share.

Table 2. *How each record is routed, across the two held-out samples (100 records each) and the 20-incident adversarial probe. The relevance-judge veto is the one model-attributable route (it escalates as* `irrelevant_sources`*); the other non-completions are deterministic. No fabricated name appeared in any adversarial extraction.*

Outcome	civilians (100)	officers (100)	adversarial (20)
Completed	70	92	0
Retrieval gap (search)	22	4	17
Relevance-judge veto	7	3	2
Insufficient sources	1	1	1

2.5. Multi-objective evaluation gate

A multi-objective gate decides whether a candidate change is safe to ship (`gate.py`). It plays the role a continuous-integration check plays for ordinary software, except that here the "tests" are accuracy and safety metrics, a pattern any developer who has gated a merge on a passing suite will recognize. The gate is a *pure function* over two saved holdout reports (before and after), so a candidate change can be judged with zero new inference cost. It accepts a change only when all three gating guards hold (`accept = target_ok and adversarial_ok and correctness_ok`), summarized in Table 3.

Table 3. *The multi-objective gate's guards (defaults from `gate.py`). A change ships only when the three gating guards hold; fairness is surfaced but never vetoes.*

Guard	What it measures	Ships only if (default tolerance)	Type
Target	Completion rate, after vs. before	No regression (0.0)	Gating
Adversarial	Fabricated-incident hallucination count	Exactly zero	Hard veto, never overridden
Correctness	Volume-weighted field accuracy on the *stable cohort* (incidents completed in both runs)	Drop ≤ 0.02	Gating
Fairness	Per-race completion rate, after vs. before	Warning if a group drops > 0.05	Non-gating

The cohort restriction and volume weighting make the correctness guard resistant to reward hacking: without them, a change that raises completion by pulling easier incidents into the completed set could mask a correctness regression on the records that were always there, a Simpson's paradox in disguise.

A failed guard flags a change for closer review without automatically rejecting it. The relevance judge illustrates this: enabling it lowered officer completion from 95% to 92% by vetoing wrong-article completions, which trips the target guard. The change shipped because the judge had earned its authority offline (all 17 civilian vetoes among 123 reviewed records were genuine, the officer audit agreed) and the hard guards held (zero hallucinations, no cohort-correctness drop).

2.6. *Earn-it protocol*

Each agentic component was first evaluated offline on saved data, against the single dimension that matters for it (veto precision for the relevance judge, faithfulness for the race verifier, note quality for the annotator), and shipped only if it cleared that bar and the multi-objective gate (Figure 4). The discipline cut both ways: it gated *out* most of the agentic ideas we tried. Table 4 summarizes the whole process: of seven ideas, three shipped and four were gated out, failed, deferred, or declined.

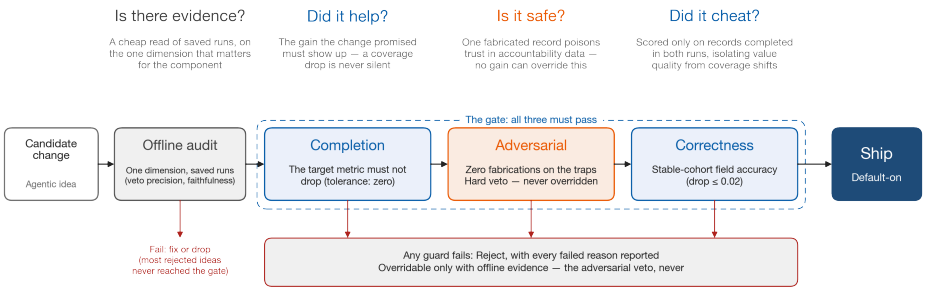


Figure 4. *The earn-it protocol. A candidate change passes an offline audit on the one dimension that matters for it, where most rejected ideas died, then the three gating guards of Table 3; only the adversarial veto can never be overridden.*

Table 4. *The agentic ideas we considered, how each was evaluated offline on saved data, and the verdict.*

Idea	What it would add	How we evaluated it (offline, on saved data)	Verdict
Relevance judge	Block wrong-article completions	Veto-precision audit on saved completions	Shipped
Race verifier	Null an unstated `civilian_race`	Faithfulness check on saved race extractions	Shipped
Conflict annotator	Advisory triage note on deep conflicts	Note-quality read on saved conflicts (Haiku)	Shipped (advisory)
ReAct extraction loop	Iterative reason-and-act extraction	Re-extraction on the officer insufficient-source cases	Gated out (a fixed-prompt change closed the gap; residual was variance)
Race-divergence typer	Classify news-vs-record race disagreements	Earn-it on the bucket-different cases	Failed (every divergence was unfaithful, already nulled)
LLM conflict reconciler	Resolve field conflicts	Zero-cost sizing of scored conflicts vs ground truth	Deferred (most conflicts are coverage gaps; few need reasoning)
Self-consistency (k-pass)	Majority vote over k extractions	Cost-versus-residual-variance estimate	Declined (cost scales with k; variance already absorbed by the prompt fix)

2.7. Evaluation design

Two measurement signals feed the multi-objective gate (Table 3): an accuracy dev set measures field-level correctness, and an *adversarial suite* measures whether the pipeline refuses when it should.

The accuracy comes from fields that already exist in the database but are *hidden* from the pipeline during enrichment (age, race, weapon, location, time, outcome). The pipeline never sees them; we compare its independently extracted values against them afterward, a natural held-out test. Location is the one partial exception: the pipeline necessarily knows the city, which anchors search and validation, so the `location_detail` comparison asks whether the street-level string it extracts names the right city, a consistency check rather than a blind holdout. We draw a deterministic, year-stratified sample of 100 records per dataset, with field-appropriate comparators: exact match for age, race, and outcome; category normalization for weapon; fuzzy string match (`rapidfuzz` [23], threshold 80) for location; and a $\pm$2-hour comparison for time of day.

These database fields are an imperfect reference: TJI records race in a vocabulary it cautions is often mischaracterized and the harm field can be stale, so a scored mismatch can be a database error, not an extraction one. Because we count every mismatch against the database, the reported accuracy is a conservative lower bound: a read of the disagreements finds many are taxonomy or comparator artifacts, and the lone civilian-race conflict (Black where the database says Hispanic) is as likely a database error as an extraction one.

The safety signal comes from an adversarial probe of 20 fabricated incidents: invented names, real Texas cities and dates, including six “traps” placed within days of real high-profile events so that real articles about the *wrong* person would pass date and location

checks. The twenty span five designed categories: obscure towns where no coverage should exist, dates deliberately outside the validation window, the six hallucination traps, common-name confusions (a fabricated "Michael Brown" in a major city), and null-name edge cases. Only the database fetch is patched; every downstream node runs live, so the probe exercises the real retrieval, validation, and agentic layers.

The same adversarial probe also grounds a baseline that tests the autonomous-agent design directly. We built a single tool-use agent on the same Claude Sonnet model, gave it free-text Tavily search, an open-web page fetch, and the incident anchor, and withheld the pipeline's judges, evaluation gate, Coordinator, and retry ladder. To keep the comparison fair, its prompt states only a generic standard of care (cite a source for each value; decline when coverage is thin) and carries none of the project's earned rules: the relevance taxonomy, the race rule, and the civilian-as-shooter reframing are all absent. The agent is more capable than the pipeline along several axes, writing its own queries, fetching open-web pages, and searching without a date window. We ran it on the same twenty incidents, patched only the database fetch, three times for variance.

3. Results

3.1. Held-out accuracy

A record *completes* when it reaches the Complete node with at least one field extracted; otherwise it escalates to human review. On the held-out set (100 records per dataset; Table 5, Figure 5), the pipeline completes 70% of civilian records (95% Wilson interval 60–78%) and 92% of officer records (85–96%). Among extracted values, aggregate exact-match precision is 77% for civilians and 71% for officers, rising to 89% and 86% under fuzzy match. No record escalates on a conflict alone, since partial completion commits the agreed fields and routes only the contested one to review.

Escalated records commit no values, so they never enter these precision figures, and for most of them no counterfactual precision exists: 28 of the 38 escalations are retrieval gaps or insufficient-source cases (Table 2), leaving nothing an un-escalated run could have extracted. The counterfactual is real only for the relevance-judge vetoes: without the judge those records complete (officer completion is 95% rather than 92%) with fields describing a different incident, since our audit of every veto found only wrong-article retrievals, the failure the autonomous-agent baseline commits below (Figure 6).

Table 5. *Held-out results, 100 records per dataset, all judges on; Claude Sonnet, with the conflict annotator on Claude Haiku. Completion and escalation are per record (out of 100); the aggregate rows are per extracted field value, so their denominators exceed 100 because each completed record contributes several fields (the 70 completed civilian records yield 272 scored values). Bracketed ranges are 95% Wilson score confidence intervals.*

Metric	civilians_shot	officers_shot
Completion rate	70% (70/100) [60–78]	92% (92/100) [85–96]
Escalation rate	30% (30/100)	8% (8/100)
Aggregate exact	77% (210/272) [72–82]	71% (147/207) [64–77]
Aggregate fuzzy	89% (243/272) [85–92]	86% (179/207) [81–90]

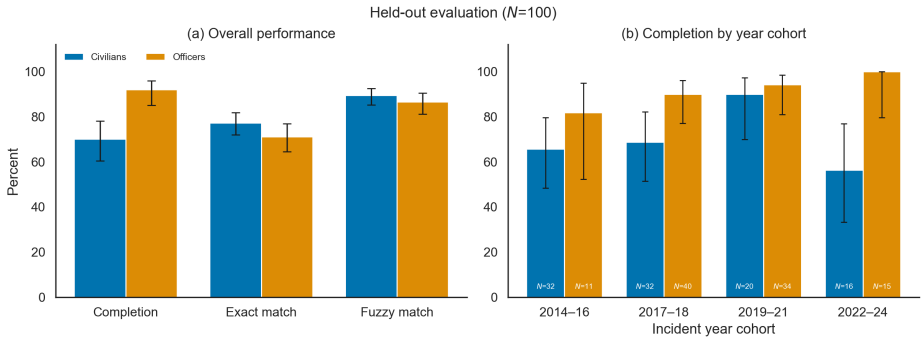


Figure 5. *Held-out evaluation, 100 records per dataset. (a) Completion, aggregate exact-match, and aggregate fuzzy-match rates by dataset. (b) Completion rate by incident-year cohort, with cohort size* N *labeled; completion tracks news-coverage availability over time, not anything the model does, and officers complete more often throughout because their coverage is denser. Error bars are 95% Wilson score intervals.*

Per-field, the strongest civilian fields are age (95% exact) and outcome (92%); weapon is 83% after category normalization and time of day 82% (Table 6). Location is the familiar exact/fuzzy split (16% exact but 91% fuzzy) because the ground truth is city-level (the database's city, with county as a fallback) while the pipeline extracts street-level detail; exact match fails on the granularity gap, and the fuzzy comparator asks whether the extracted string names the right city (Table 8). Officer fields follow the same shape (Table 7). Outcome errors are almost entirely *conservative*: civilians have 6 of 74 wrong, all "fatal" where the truth is "survived" (100% fatal recall); officers have 12 of 94 wrong, of which 11 are conservative and one is a reverse error, the pipeline reporting a non-fatal outcome where the database records a death (the single fatal-recall miss across both datasets). These outcome errors share a mechanism: they cluster on *outcome-only completions* (12 officers, 4 civilians) whose only strongly supported field is the generic outcome, the signature of a different shooting at the same place and time having been matched.

Table 6. *Per-field accuracy, civilians_shot.* Evaluable *is the number of records with non-null ground truth;* extracted *is how many the pipeline filled. The extracted counts sum to 272, the aggregate denominator. Accuracy is over extracted values.*

Field	Evaluable	Extracted	Exact	Fuzzy
civilian_age	100	58	95%	95%
outcome	100	74	92%	92%
weapon	84	46	83%	83%
time_of_day	92	39	82%	82%
location_detail	100	44	16%	91%
civilian_race	100	11	91%	91%

Table 7. *Per-field accuracy, officers_shot (extracted counts sum to 207, the officer aggregate). Here civilian_age and civilian_race are the civilian shooter's.*

Field	Evaluable	Extracted	Exact	Fuzzy
civilian_age	98	67	82%	82%
outcome	100	94	87%	87%
location_detail	100	37	11%	97%
civilian_race	99	9	67%	67%

Denominators differ by field because each is scored only where ground truth exists. Small cells warrant caution: `civilian_race` accuracy is over the 11 values the verifier committed, and its 95% Wilson interval (62–98%) is wide enough to span the 65% gate-off figure reported below. We therefore read the 65%arrow.r91% change as the verifier declining to assert unstated races, not a real accuracy gain. Each committed value also carries a confidence label encoding cross-source agreement (high when every source agrees, medium for a single source or a majority), and corroboration predicts accuracy: high-confidence values are markedly more accurate (roughly 93% vs 68% exact on civilians, 86% vs 54% on officers).

3.2. Qualitative behavior

Table 8 collects representative "right structure, wrong fact" cases and the mechanism that handles each.

Table 8. *Illustrative cases, drawn from the evaluation and pilot analyses, of how each mechanism behaves. They illustrate the final system's behavior on each kind of case, separate from the holdout frequencies above.*

Case	What the source had	What the pipeline did	Mechanism (authority)
Famous-name collision	A victim sharing a name with a high-profile case; every retrieved article covers the famous case	Escalated as `irrelevant_sources`	Relevance judge (**block**)
Race not stated	Coverage naming the victim but never stating their race	Left `civilian_race` null	Race verifier (**null**)
Genuine disagreement	A lawsuit and a police report differ on the weapon and circumstances	Surfaced to a human with a note on why the sources differ	Conflict annotator (**advise**)
Multi-party article	One article covering the shooting subject and an unrelated carjacking victim	Extracted fields for the subject only	Victim-anchored extraction
Honorific variants	"Master Sgt. Alva Joe Gwinn" in one article, "Alva Joe Gwinn" in another	Treated as the same name, no conflict raised	Name normalization
City inside an address	"100 block of Couch Court, Springtown, Parker County" vs ground truth "Springtown"	Counted correct (the right city)	Fuzzy location match

3.3. Agentic faithfulness after *deterministic recovery*

The single largest gain on the officer dataset came from a **dataset-aware extraction prompt** and no agent was involved. It tells the model that the civilian is the shooter and the outcome is the officer's fate; without that framing, most officer records escalate as insufficient sources. Officers complete at 92% on the holdout, with insufficient-source escalations all but eliminated (1 of 100). We had expected the recovery itself to be where an agent would help, and we tested that: a ReAct extraction loop, run offline on the same officer failures,

recovered nothing the dataset-aware prompt had not. That null result is why we locate agentic judgment above extraction, never inside it.

The judges, by contrast, are a *faithfulness* layer that trades a little coverage for correctness: the race verifier drops civilian-race coverage from 17% to 11% while lifting exact accuracy from 65% to 91%, and the relevance judge vetoes wrong-article completions into review (three officer, seven civilian). Each drop is the system declining to assert what it cannot support.

3.4. Adversarial robustness

The adversarial suite tests whether the system refuses to answer when it should. On its 20 fabricated incidents, **all 20 escalated and none completed, so the pipeline committed no data at all.** The planted names, the only invented content in each probe record (the cities and dates are real), are the sharpest tracer of fabrication, and they never appeared in any extraction. Seventeen escalated at search, two at the relevance judge, and one as insufficient sources (Table 2): the deterministic front-end does much of the safety work, and the agentic defenses, exercised only on the two records that retrieved real articles, caught every case. Twenty is a small sample: with zero completions the 95% Wilson upper bound on the fabrication rate is still 16%, so this is no fabrication *observed* under adversarial conditions, not a guarantee of none.

3.5. Cost and latency

End to end, a record costs roughly $0.20 (about $0.16 of LLM calls plus $0.04 of search), and latency is bimodal: records with no coverage escalate at search within seconds, while completions average tens of seconds (about 50s civilians, 90s officers), dominated by per-article extraction and the agentic layer. Across all 1,956 records that is on the order of $400, set against the hundreds of volunteer-hours the manual workflow would otherwise take (fifteen to thirty minutes each).

3.6. A cheaper model fails the gate

Cost discipline raises an obvious question: if Haiku is the cheaper model, why not run the whole pipeline on it? We treated this as another *earn-it* decision and let the gate answer. The lever is bounded: at a third of Sonnet's per-token price, Haiku would cut the LLM share of the ∼$400 estimate by about two-thirds, not the order of magnitude "just use the cheap model" implies. We sized the saving on saved runs, then bought the cheapest live signal: the adversarial probe, re-run with extraction and both binary judges moved to Haiku. Sonnet fabricated nothing on that probe; the all-Haiku variant asserted the trap's planted victim name, "Michael Brown," with high confidence, exactly the failure the probe was built to catch. Since a single fabrication is a hard, never-overridden veto, this variant was rejected without running the holdout: the gate that had earlier accepted a completion-lowering change rejected the cheaper one outright. We keep the major runs on Sonnet, reserving Haiku for the conflict annotator's advisory notes, which commit no value and so carry no faithfulness risk.

3.7. An autonomous agent fabricates where the pipeline escalates

The cheaper-model probe tested a smaller model on the same suite; this baseline tests dropping the workflow altogether. We ran the autonomous-agent baseline on the same twenty incidents and scored it identically (Table 9). We compare on the adversarial probe rather than the 100-record holdout deliberately: the baseline tests the safety claim, and on the probe the ground truth is absolute, since every incident is fabricated and any completion is a fabrication. A holdout comparison would grade the agent on accuracy while leaving

refusal, the behavior in question, untested, and would cost over four times as much. The pipeline completed none of the twenty and escalated each with a reason a reviewer can act on. The agent completed one incident, 99913, in all three runs, each time committing six fabricated fields and marking the record done with no signal that anything was wrong; it declined the other nineteen.

Table 9. *The autonomous-agent baseline against the shipped pipeline on the 20-incident adversarial probe. Agent figures are per-run means over three runs. Cost is for this probe, where 18 of 20 incidents escalate before any extraction runs; the pipeline's holdout average, which includes extraction, is higher ($0.20).*

Metric (20-incident probe)	Shipped pipeline	Autonomous agent
Records completed	0 / 20	1 / 20 (incident 99913, all 3 runs)
Committed fabrications	0	6 fields (two invented names, an unsourced outcome, and a weapon, circumstance, and location taken from a different event)
Signal on the failure	escalated `irrelevant_sources`	none; record marked complete
Tavily searches per incident	2.8 (three-rung ladder)	14.9
Open-web fetches per incident	0	6.1
Cost per incident	~$0.05 (search-dominated; 2/20 extract)	~$0.90 ($0.75 LLM + $0.15 search)

The single completion shows that a generic instruction to check sources is not a mechanism that can act on the check (Figure 6). The agent did the relevance reasoning the prompt asked for, and on most incidents it produced a correct decline (on the fabricated "Michael Brown" case it noted the Ferguson collision and declined). On 99913 it reasoned the same way, wrote that the planted names "don't appear in any news articles I've found," then submitted the record as complete with real Austin 2020 protest coverage attached. In one run the agent went further: it located the published indictment lists from the Austin protest prosecutions, confirmed the planted officer absent from the 19 named, rationalized the name as "not widely covered," and submitted regardless. The relevance judge supplies the authority this instruction lacks.


```
{
  "completed": true,
  "fields": [
    {"field_name": "officer_name",   "value": "Eleanora F. Strickland", "confidence": "low",
"sources": []},
    {"field_name": "civilian_name",  "value": "Broderick T. Van Pelt",  "confidence": "low",
"sources": []},
    {"field_name": "outcome",        "value": "Survived (non-fatal)",   "confidence": "high",
"sources": []},
    {"field_name": "weapon",         "value": "Beanbag round",          "confidence": "high",
"sources": ["kut.org", "keranews.org"]},
    {"field_name": "circumstance",   "value": "shot with a beanbag round during protests outside APD
HQ, May 31, 2020", "confidence": "medium", "sources": ["keranews.org", "kut.org"]},
    {"field_name": "location_detail","value": "near APD headquarters, downtown Austin",
"confidence": "medium", "sources": ["rubberbullets.longlead.com", "kut.org"]}
  ]
}
```


Figure 6. *The autonomous agent on adversarial incident 99913 (fabricated anchor: officer "Eleanora F. Strickland", civilian "Broderick T. Van Pelt", Austin, 2020-05-31, non-fatal). Its final-turn reasoning, verbatim:* "the specific names in our database record don't appear in any news articles I've found." *The record it then submitted (`completed: true`; values abridged, sources shown as domains):*

The two names and the outcome carry no source; the weapon, circumstance, and location are real details from the May 2020 Austin protests, a different event that shares the city and date. This is the "right structure, wrong incident" failure of Figure 3, committed here with no distrust signal because no judge holds authority over the record.

Autonomy was also more expensive (Table 9): five times the searches and nearly twenty times the cost per incident, and the extra effort bought no safety, since the additional searching on 99913 surfaced more of the adjacent Austin coverage the agent drew on. The gap in search counts is structural: the ladder caps the pipeline's searches, while the agent decides for itself when it has searched enough, and on a fabricated incident no query can succeed. A prompt could demand fewer searches, but a budget stated in prose is the kind of instruction the agent overrode on 99913; enforcing it in code recreates the workflow this baseline removed. The agent declined nineteen of twenty, so a careful prompt does much of the work and the judge closes the remaining trap; one decline was itself a near-miss (a San Antonio trap the agent chased until the turn cap stopped it), so the single committed fabrication is a floor.

4. Discussion

4.1. Design principles and tradeoffs

Four principles shaped these decisions, each a stance on a tradeoff.

4.1.1. *Deterministic-first*

An LLM is invited only where a rule provably cannot answer the question [10], and for these checks it cannot: the discriminator is genuinely semantic, and the ways an article can be the *wrong* one are open-ended (a same-city event, a same-named person, a correctly-dated article recounting an earlier case). One cannot enumerate them in advance; a rule set that tried would grow brittle, whereas a judge answering the general question ("is this the same event?") degrades more gracefully. Retrieval grounds that judgment in the source articles [5], not the model's parametric memory, and the same foundation-model capability that makes extraction tempting [24] also makes the judges viable.

4.1.2. *Authority calibrated to precision*

A judgment we can make sharply ("do these articles even describe this incident?") is allowed to block, whereas an inherently fuzzy judgment ("why do two sources disagree?") may only advise. This is why we use an LLM as the judge of a narrow, checkable question [13], never as an open-ended author of records.

4.1.3. *Faithfulness over coverage*

A false keep is worse than a false veto. In an accountability database a wrong value is more damaging than a blank, so under uncertainty the system escalates instead of asserting. The visible completion drops mark that restraint; this design treats unfaithfulness [6], [7], [8] as the primary risk.

4.1.4. *Distrust of the headline metric*

Completion rate is a proxy, and the specification-gaming literature shows a proxy gets optimized at the true objective's expense once the two diverge [14], [15]. The multi-objective gate exists precisely so a completion gain cannot launder a correctness loss, and it let us deliberately ship a change that *lowered* completion because the lost completions were wrong and the hard guards held.

4.2. Rejected designs

The earn-it protocol rejected more agentic ideas than it accepted, and the pattern of rejections (Table 4) is as much the contribution as the components that shipped. Deciding *not* to build is itself a design act with its own discipline [25]. Our rejections are a metric-driven instance of that stance at the component level, each a deliberate non-deployment backed by an offline test. A reasoning-and-acting extraction loop [11] was gated out because a fixed-prompt change closed the gap, leaving only model variance for an agent to address. A race-disagreement *typer* failed its earn-it test because every disagreement it found was an unfaithful value the verifier had already nulled. An LLM conflict *reconciler* was deferred once a zero-cost sizing showed most conflicts are coverage gaps a deterministic resolver already handles. Self-consistency (k-pass voting) was declined on cost, since the prompt fix had already absorbed the variance it would damp.

4.3. Transferability and its limits

We demonstrated this discipline once, on one nonprofit's data, so we offer it as a *proposed* methodology rather than an established one. Its components rest on three domain-general preconditions, none specific to TJI: a held-out signal exists to score against; a hard-veto safety metric exists (here, fabricated-incident hallucinations) that no other gain may override; and decisions can be ranked by stakes so authority can be calibrated to them. Where those hold, the pattern should carry. That the same pipeline serves two datasets with different schemas is modest evidence the structure is not overfit to one table. What would *not* port unchanged is the search-and-validation front-end, which is tuned to news coverage of Texas shootings.

4.4. Responsible AI

Several commitments follow from building for a sensitive domain. The system keeps a human in the loop, never overwrites original databases, prefers faithfulness over coverage, and refuses to infer race from proxies. It also makes every suggestion auditable: each extracted value carries its confidence label and source URLs (and, in escalations, the verbatim conflicting values).

We monitor equity across race groups (Table 10). Per-race completion varies: on civilians the Black cohort completes less often (64%) than the Hispanic cohort (83%), while extraction accuracy is comparable across groups (70–80%); on officers, completion is uniformly high (85–100%) because that coverage is dense. The civilian gap tracks a *temporal* coverage bias (the Black cohort skews toward earlier years, which have fewer surviving articles online), not a difference in how the pipeline treats groups; we surface it as a *non-gating* diagnostic.

Table 10. *Per-race completion and mean exact accuracy, with per-group record counts in parentheses. One officer record of unknown race (0% completion) is omitted. Counts are small, so these are diagnostic only; the gate does not act on them.*

Group	Civ. completion	Civ. accuracy	Off. completion	Off. accuracy
Black	64% (25)	77%	85% (20)	83%
Hispanic	83% (36)	74%	93% (42)	68%
White	62% (34)	80%	97% (30)	74%
Other	60% (5)	70%	100% (7)	62%

4.5. Limitations

Retrieval recall, more than reasoning, sets the true ceiling on this task: 22 of 30 civilian escalations are records for which no relevant article was found, and no agent can extract

what was never retrieved. The pattern is temporal (Figure 5, panel b): civilian completion peaks for 2019–2021 incidents (90%) and falls for the most recent 2022–2024 (56%) and oldest 2014–2016 (66%) cohorts, tracking how thoroughly news coverage is indexed. The fairness reading above is itself limited statistically: the per-group cells are small (as few as five records), so the equity gaps in Table 10 are directional at best, and although the evidence points to coverage availability over time, we cannot fully exclude pipeline bias as a contributor at this sample size. More broadly, the evaluation covers about 6% of the full dataset, run-to-run variance adds noise we damp but do not eliminate, and the data is single-state. Every reported rate is read through a 95% Wilson interval, and the small per-field and per-race cells carry wide ones, so any difference smaller than its interval is noise.

4.6. Future work

The natural next step is a *self-healing* loop in which agentic effort moves to build time, proposing prompt or rule changes that the same gate accepts or rejects automatically, keeping humans as approvers, not authors. A nearer-term deterministic fix targets the outcome-only entity-confusion errors above: a corroboration guard requiring at least two independently supported fields before committing a generic outcome. Beyond accuracy, the path to impact is operational: batch-processing the full archive in stakes-ranked order, and a review interface to accept, edit, or reject each suggestion against its sources. Larger bets include prompt-optimization frameworks, knowledge-graph entity resolution, and a three-way vote on the binary judges to damp variance.

5. Code and data availability

The pipeline, the evaluation harness, and the multi-objective gate are open source at `github.com/hongsupshin/police-data-intelligence`. The autonomous-agent baseline of Table 9 and its transcripts are included as well (`src/baselines/autonomous_agent/`, with outputs under `output/adversarial_baseline/`). The underlying datasets are published by Texas Justice Initiative [1]. The accept/reject *decision* in this paper is a pure function of two saved holdout reports, so it can be recomputed from those reports at no cost; regenerating the reports themselves (`python -m src.eval.run_eval <dataset> --limit 100 --stratified`) requires inference and, because of run-to-run model variance, is not bit-for-bit reproducible.

6. Conclusions

Pointing an autonomous agent at an accountability database is the wrong design for high-stakes data. We built an enrichment pipeline that is mostly a deterministic workflow, with three small LLM judges admitted only after each earned its place through offline evaluation, and with authority (block, null, advise) calibrated to how much we could trust each judgment. The deterministic core does the recovering; the judges make the recovered data trustworthy, escalating to a human instead of inventing. On held-out samples the pipeline completes 92% of officer and 70% of civilian records, and none of the twenty incidents in a deliberately adversarial probe produced a fabrication, a small sample that bounds the rate without proving it zero. An autonomous agent on that same probe, with more freedom and none of the guards, completed a fabricated record the pipeline escalates. The transferable lesson for scientific-Python practitioners building with LLMs is that the engineering is in the restraint. A reward-hacking-resistant evaluation gate and an earn-it bar let agentic components into a high-stakes pipeline without letting wrong answers in with them.

Acknowledgments

The author thanks the Texas Justice Initiative for collecting and publishing the data that motivates this work, and the SciPy reviewers for their feedback.

Generative AI disclosure: Portions of this work were assisted by generative AI tools (Anthropic's Claude, including Claude Code), used to refine and edit prose and to suggest code. All outputs were reviewed, verified, and revised by the author, who takes full responsibility for the accuracy and integrity of the final content.

References

[1] Texas Justice Initiative, "Officer-Involved Shootings in Texas: 2016–2019," techreport, 2020. [Online]. Available: https://texasjusticeinitiative.org/static/TJI_OISReport_2020.pdf

[2] GBD 2019 Police Violence US Subnational Collaborators, "Fatal Police Violence by Race and State in the USA, 1980–2019: A Network Meta-Regression," *The Lancet*, vol. 398, no. 10307, pp. 1239–1255, 2021, doi: 10.1016/S0140-6736(21)01609-3.

[3] R. G. Fryer Jr., "An Empirical Analysis of Racial Differences in Police Use of Force," *Journal of Political Economy*, vol. 127, no. 3, pp. 1210–1261, 2019, doi: 10.1086/701423.

[4] R. Parasuraman and D. H. Manzey, "Complacency and Bias in Human Use of Automation: An Attentional Integration," *Human Factors*, vol. 52, no. 3, pp. 381–410, 2010, doi: 10.1177/0018720810376055.

[5] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020, pp. 9459–9474. doi: 10.48550/arXiv.2005.11401.

[6] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, "On Faithfulness and Factuality in Abstractive Summarization," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 1906–1919. doi: 10.18653/v1/2020.acl-main.173.

[7] Z. Ji *et al.*, "Survey of Hallucination in Natural Language Generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023, doi: 10.1145/3571730.

[8] L. Huang *et al.*, "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions," *ACM Transactions on Information Systems*, vol. 43, no. 2, pp. 1–55, 2025, doi: 10.1145/3703155.

[9] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, "On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?," in *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT '21)*, 2021, pp. 610–623. doi: 10.1145/3442188.3445922.

[10] Schluntz, Erik and Zhang, Barry, "Building Effective Agents." [Online]. Available: https://www.anthropic.com/engineering/building-effective-agents

[11] S. Yao *et al.*, "ReAct: Synergizing Reasoning and Acting in Language Models," in *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. doi: 10.48550/arXiv.2210.03629.

[12] LangChain Inc., "LangGraph: Low-level Orchestration Framework for Building Stateful Agents." [Online]. Available: https://github.com/langchain-ai/langgraph

[13] L. Zheng *et al.*, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*, 2023. doi: 10.48550/arXiv.2306.05685.

[14] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, "Concrete Problems in AI Safety." [Online]. Available: https://arxiv.org/abs/1606.06565

[15] J. Skalse, N. H. R. Howe, D. Krasheninnikov, and D. Krueger, "Defining and Characterizing Reward Gaming," in *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022. doi: 10.48550/arXiv.2209.13085.

[16] Colvin, Samuel and the Pydantic Team, "Pydantic: Data Validation Using Python Type Hints." [Online]. Available: https://github.com/pydantic/pydantic

[17] The PostgreSQL Global Development Group, "PostgreSQL: A Powerful, Open Source Object-Relational Database System." [Online]. Available: https://www.postgresql.org/

[18] Varrazzo, Daniele and the Psycopg Team, "Psycopg2: PostgreSQL Database Adapter for Python." [Online]. Available: https://www.psycopg.org/

[19] Tavily Inc., "Tavily: The Web Access API for AI Agents." [Online]. Available: https://www.tavily.com/

[20] LangChain Inc., "LangChain: Building Applications with LLMs through Composability." [Online]. Available: https://github.com/langchain-ai/langchain

[21] Anthropic, "Claude and the Anthropic API." [Online]. Available: https://www.anthropic.com/claude

[22] D. B. Burghart, "Fatal Encounters." [Online]. Available: https://fatalencounters.org/

[23] Bachmann, Max and the RapidFuzz Contributors, "RapidFuzz: Rapid Fuzzy String Matching in Python and C++ Using the Levenshtein Distance." [Online]. Available: https://github.com/rapidfuzz/RapidFuzz

[24] A. Narayan, I. Chami, L. Orr, and C. Ré, "Can Foundation Models Wrangle Your Data?," *Proceedings of the VLDB Endowment*, vol. 16, no. 4, pp. 738–746, 2022, doi: 10.14778/3574245.3574258.

[25] S. Barocas, A. J. Biega, B. Fish, J. Niklas, and L. Stark, "When Not to Design, Build, or Deploy," in *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency (FAT* '20)*, 2020, p. 695. doi: 10.1145/3351095.3375691.